Grid Features Validation Packet -- XB2IQX

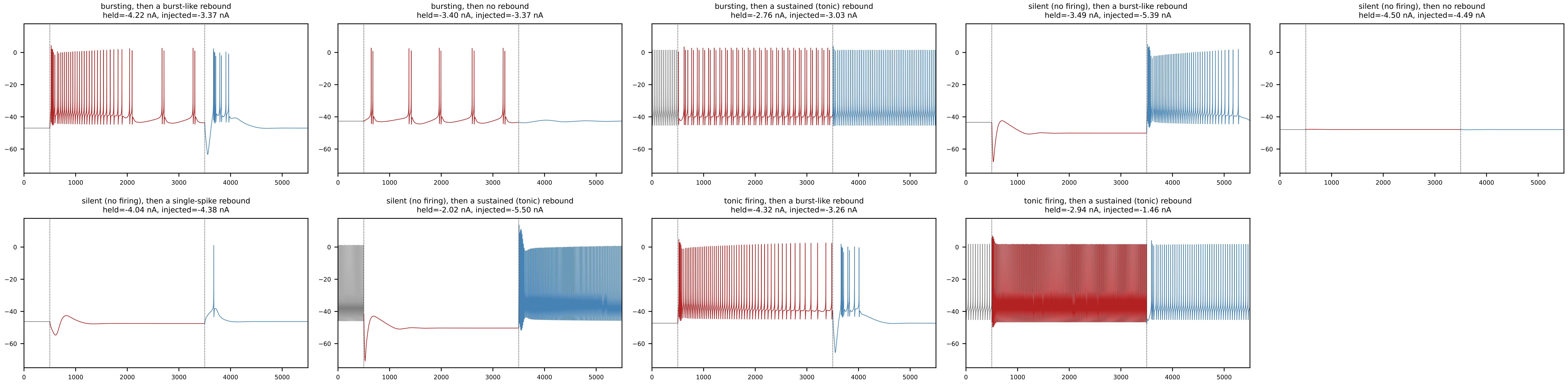
Every trace in this packet is simulated output from a fitted, multi-conductance single-compartment model of this cell, integrated deterministically -- not a raw current-clamp recording. What is being evaluated here is the feature-extraction methodology applied on top of that model: spike detection, burst/tonic classification, post-inhibitory rebound detection, sag depth, and spike-frequency adaptation. Each figure below reruns the actual classification algorithm on a real simulated trace and marks what it found directly on the trace, so each classification can be checked by eye rather than taken on faith.

```
Silencing threshold: -4.50 nA   |   2500 points sampled across the current grid
Burstiness index: 0.316
Firing-rate/current slope: 18.44 Hz/nA (R^2=0.962)
```

Contents:

python src/generate_validation_packet.py
source: cell_held_injected_grid.pkl + cell_grid_features.pkl

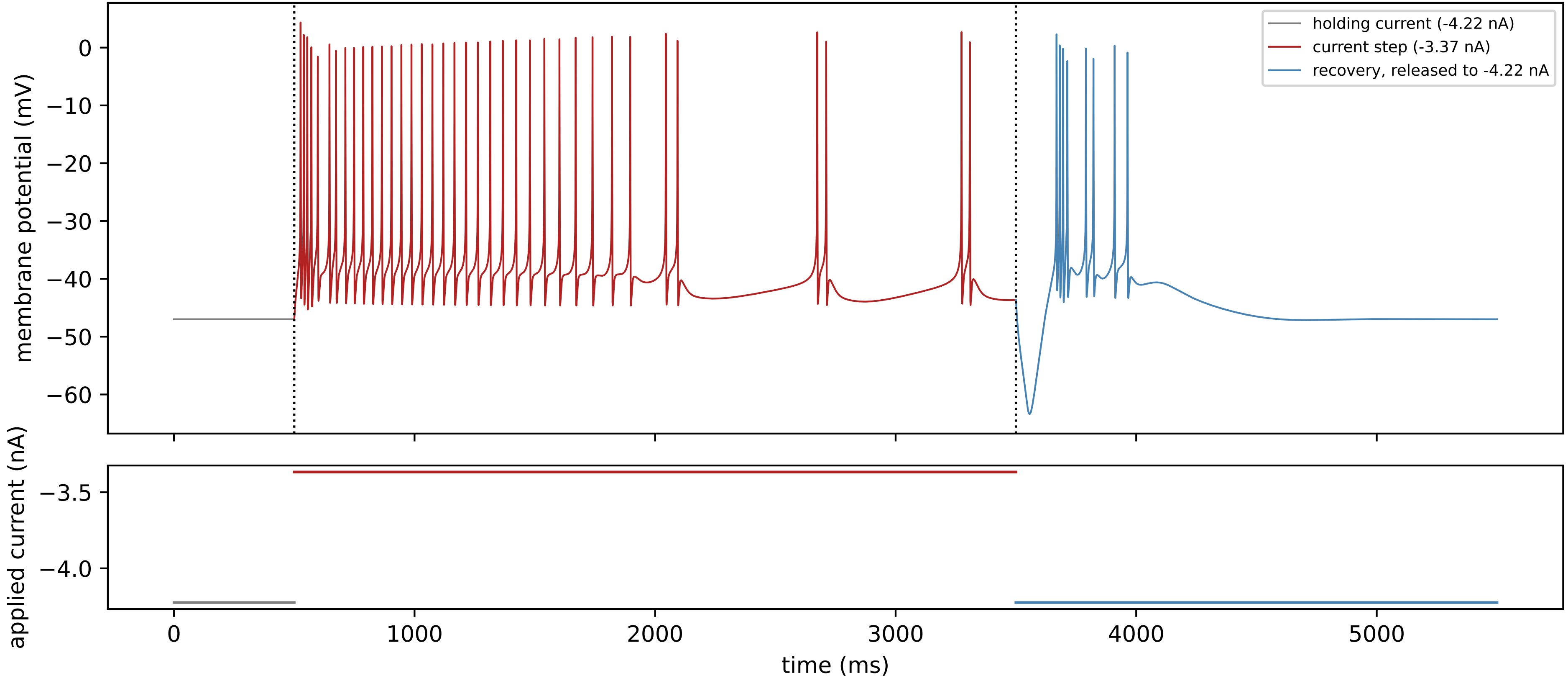
1. Representative traces



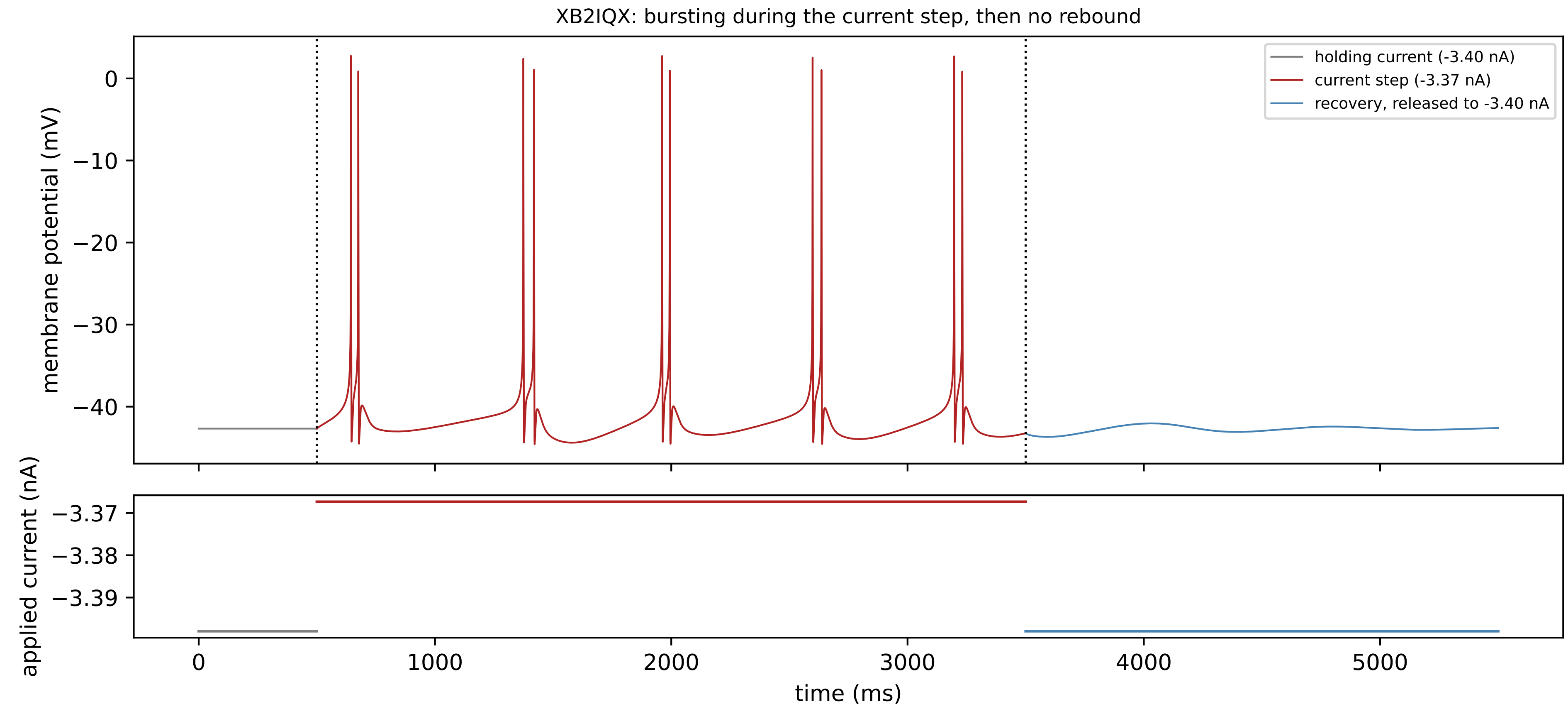
One representative trace for each distinct combination of response-to-current-step pattern and rebound pattern actually observed across this cell's full 2500-point grid of held and injected current levels: gray is the held baseline before the current step, red is the response during the current step, and blue is the recovery period after release back to the held level. Each combination is represented by its most clearly-expressed example among the grid points that produced it, not a hand-picked illustration. Every panel shares the same voltage (-75 to 18 mV) and time (0-5500 ms) axes so amplitude and duration are directly comparable by eye -- without this, a small subthreshold wobble and a full-amplitude spike train would each be independently rescaled to fill the same panel and look equally prominent.

```
python src/generate_validation_packet.py
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point()
```

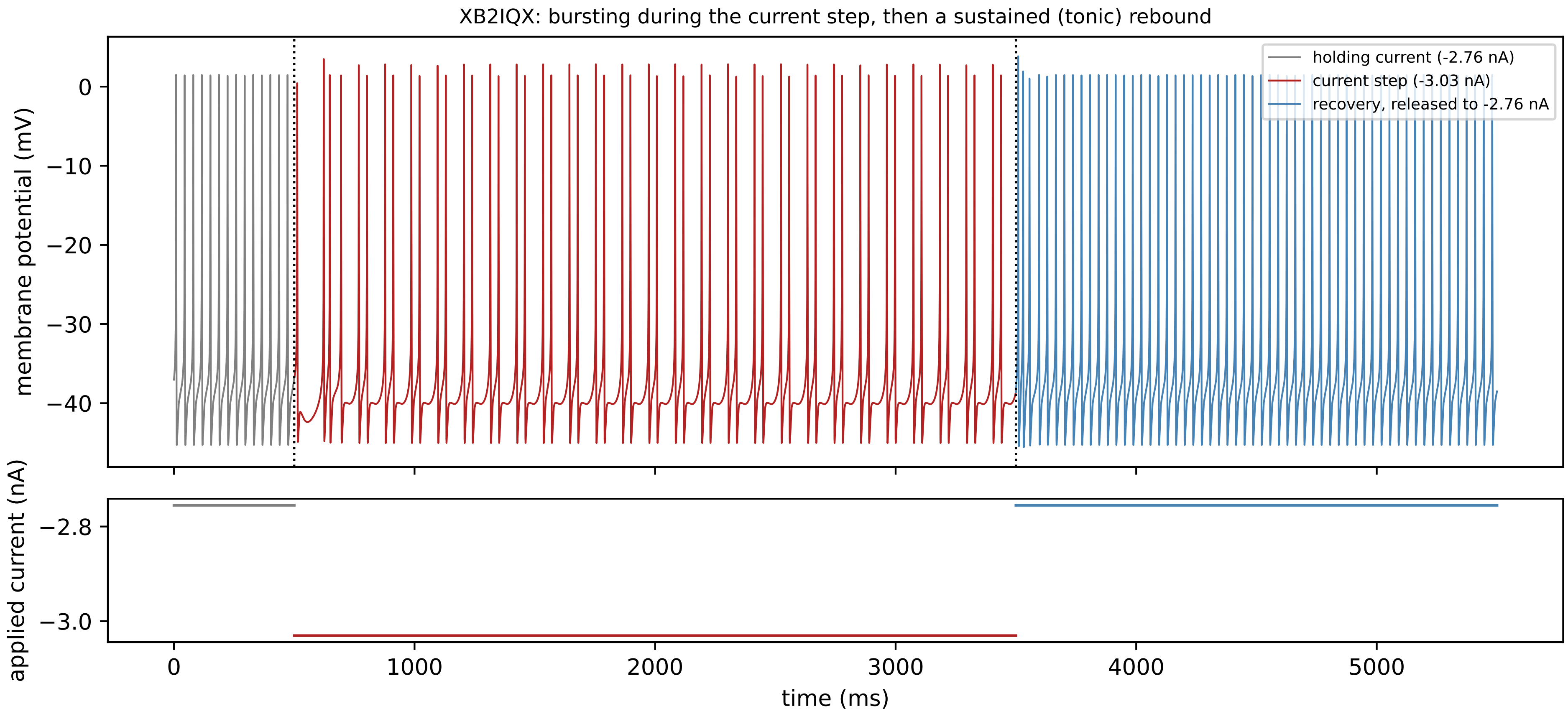
XB2IQX: bursting during the current step, then a burst-like rebound



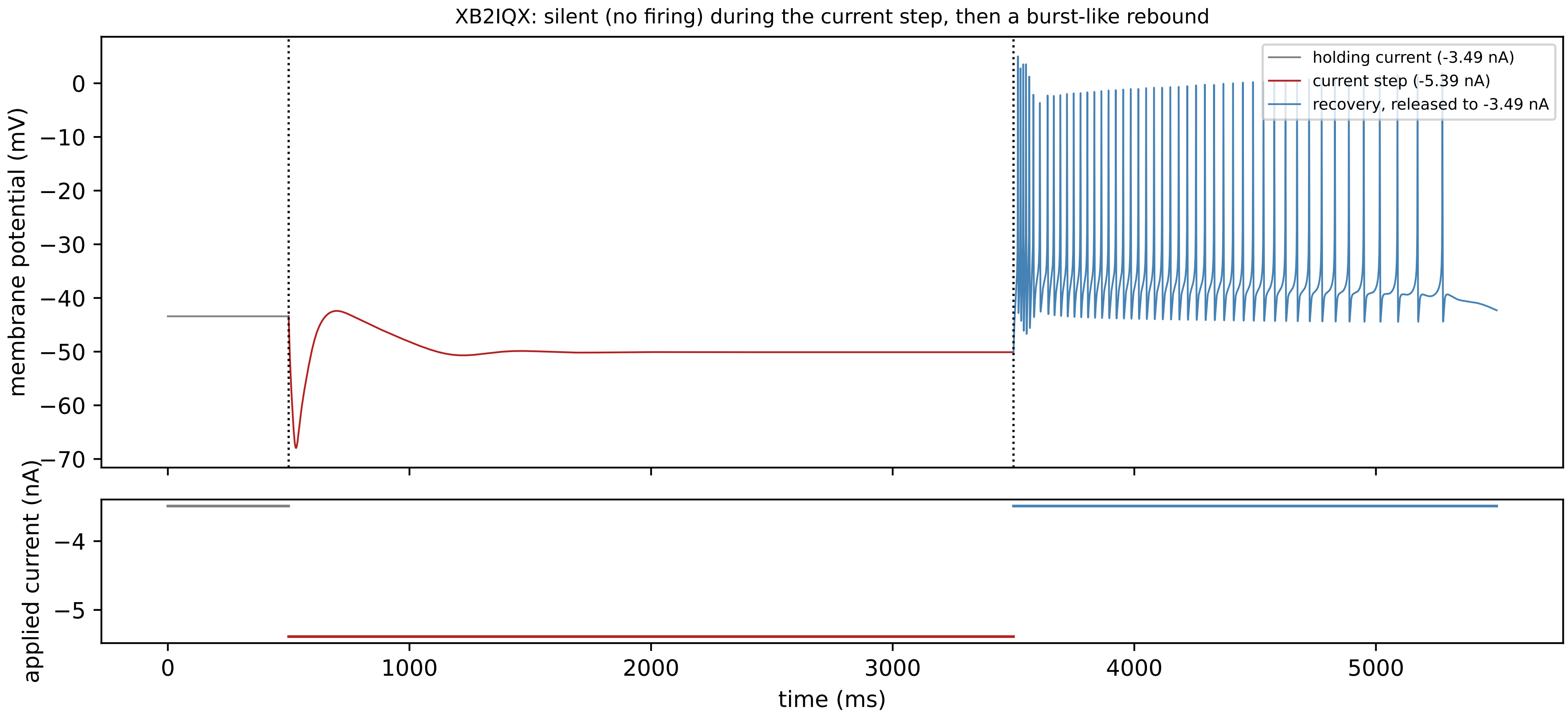
python src/generate_validation_packet.py
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()



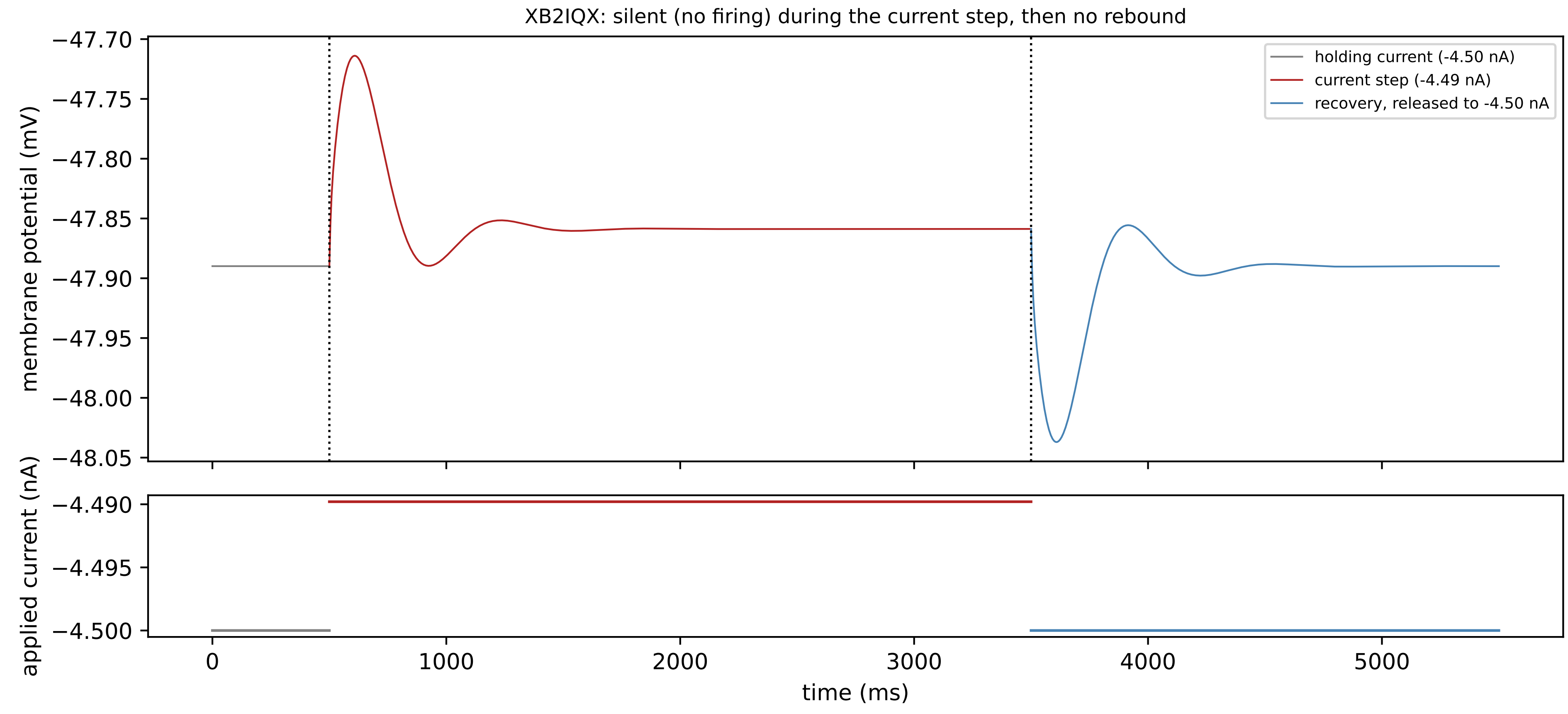
python src/generate_validation_packet.py
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()



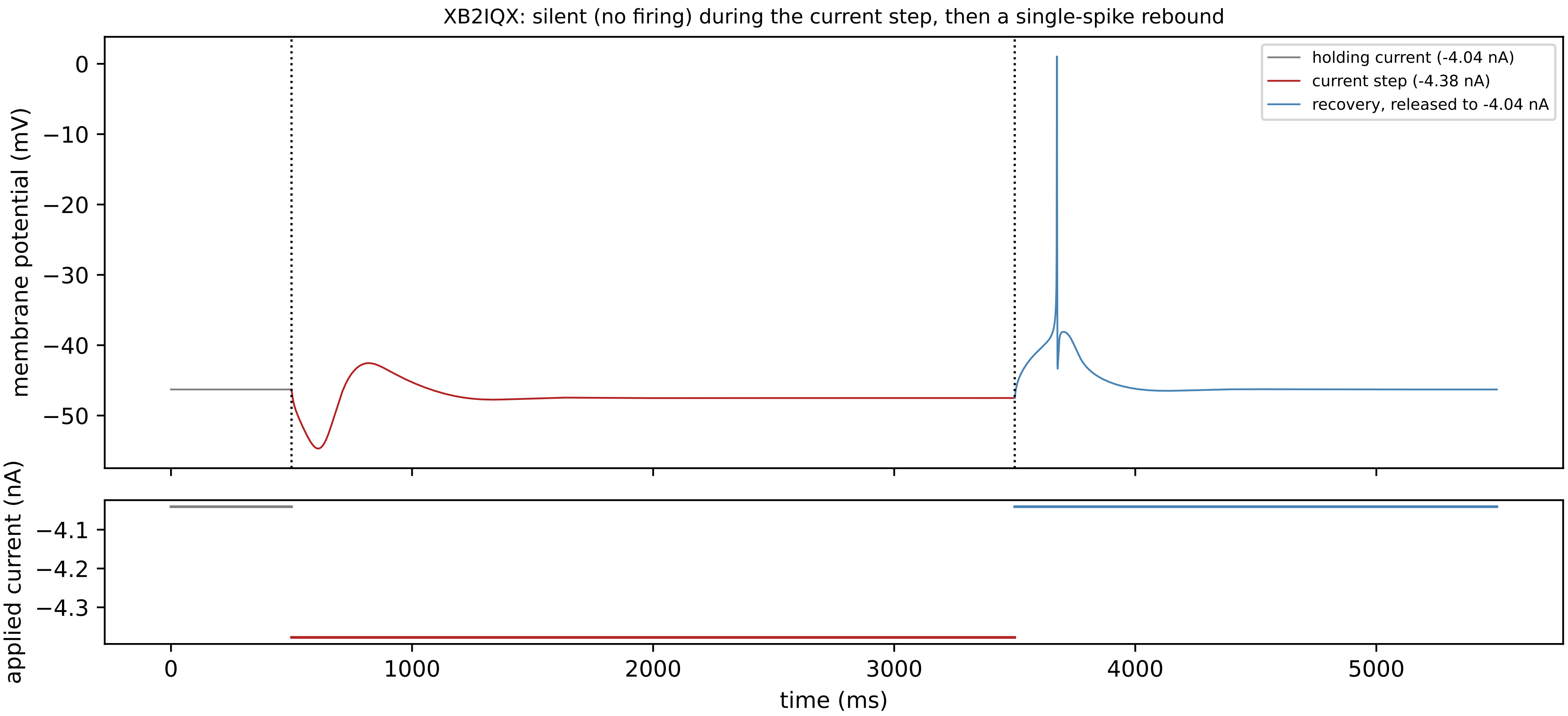
python src/generate_validation_packet.py
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()



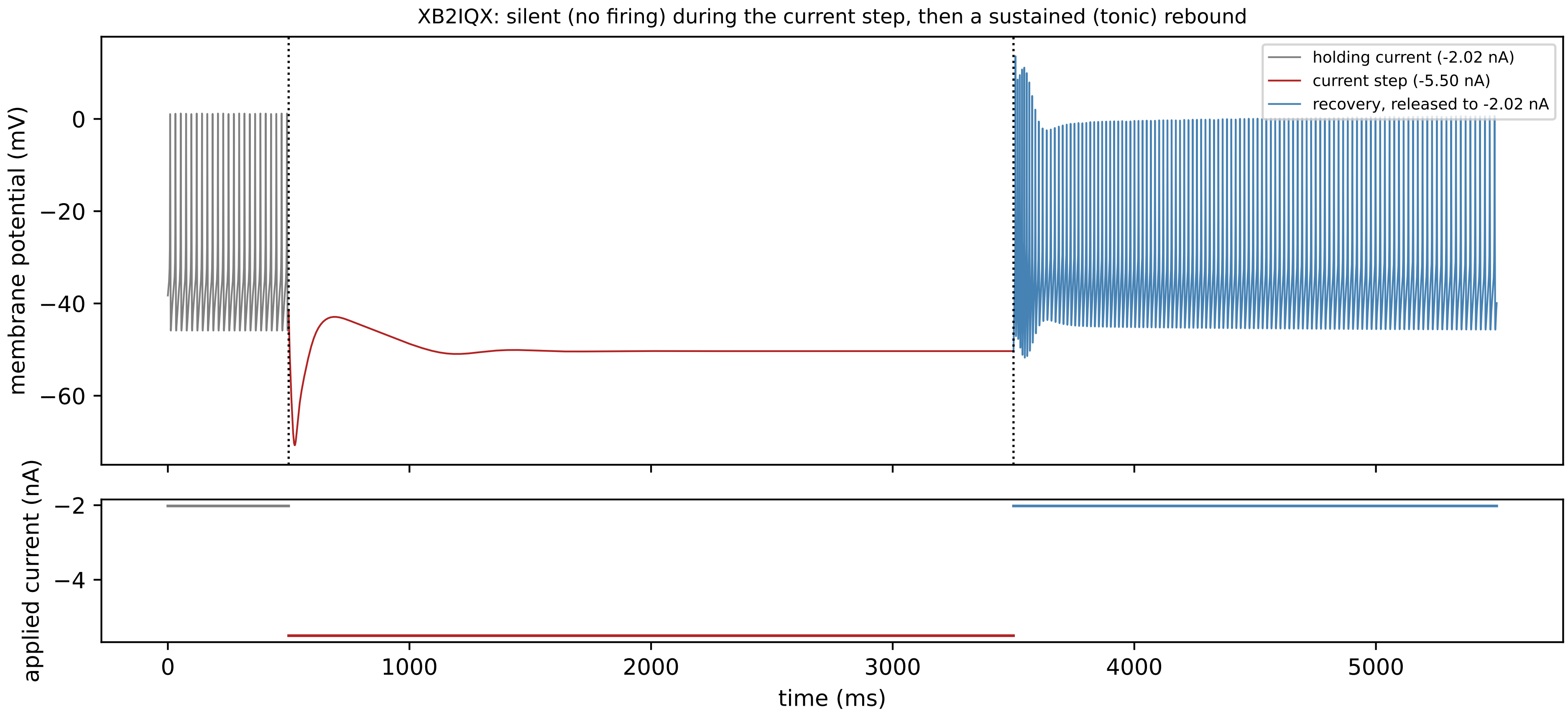
python src/generate_validation_packet.py
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()



python src/generate_validation_packet.py
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()

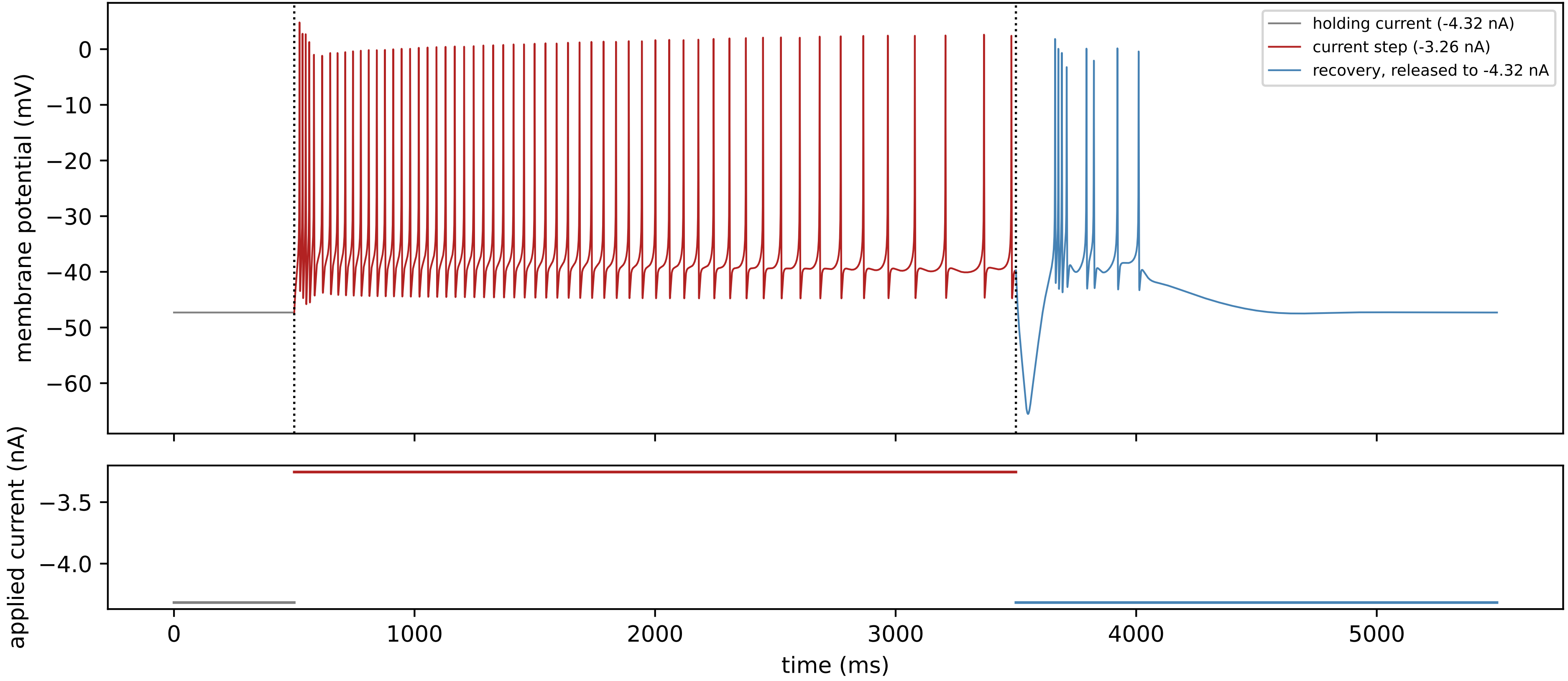


python src/generate_validation_packet.py
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()



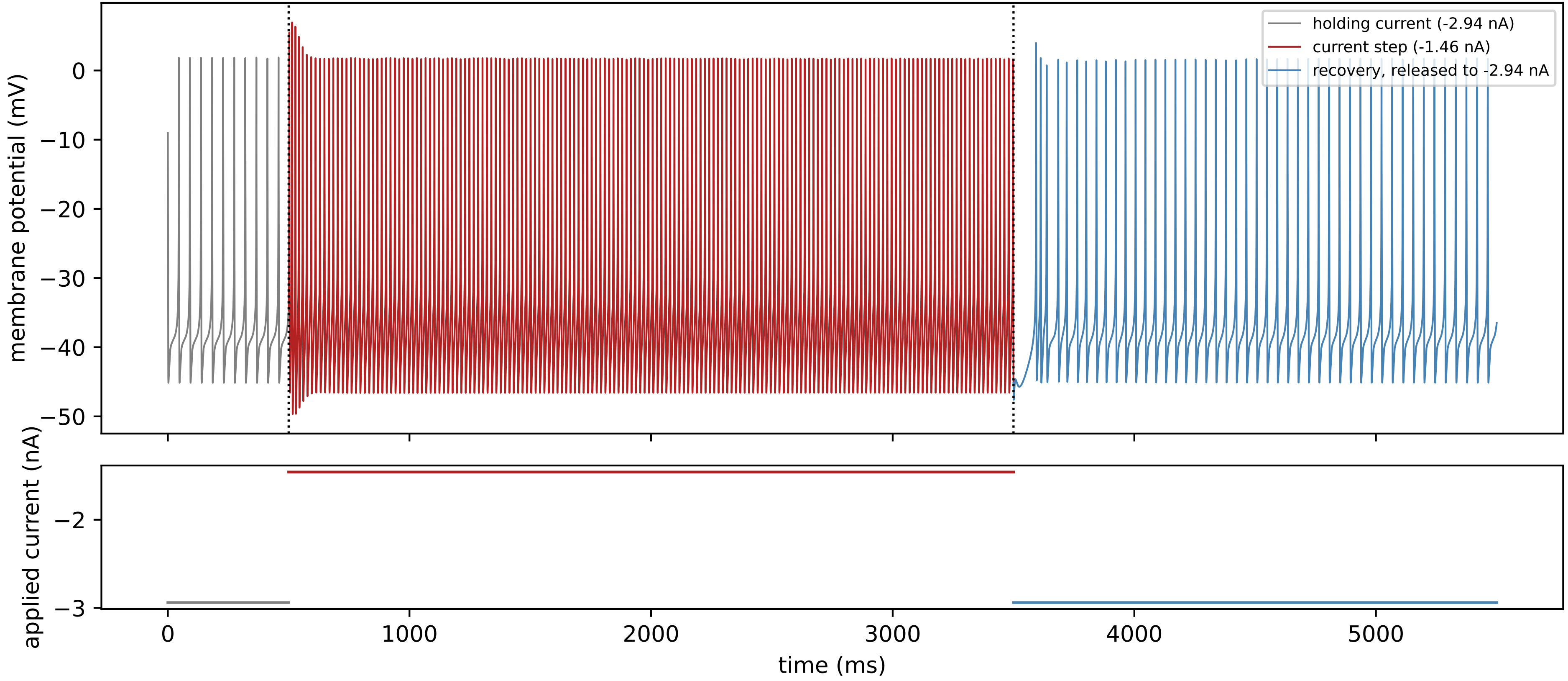
python src/generate_validation_packet.py
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()

XB2IQX: tonic firing during the current step, then a burst-like rebound



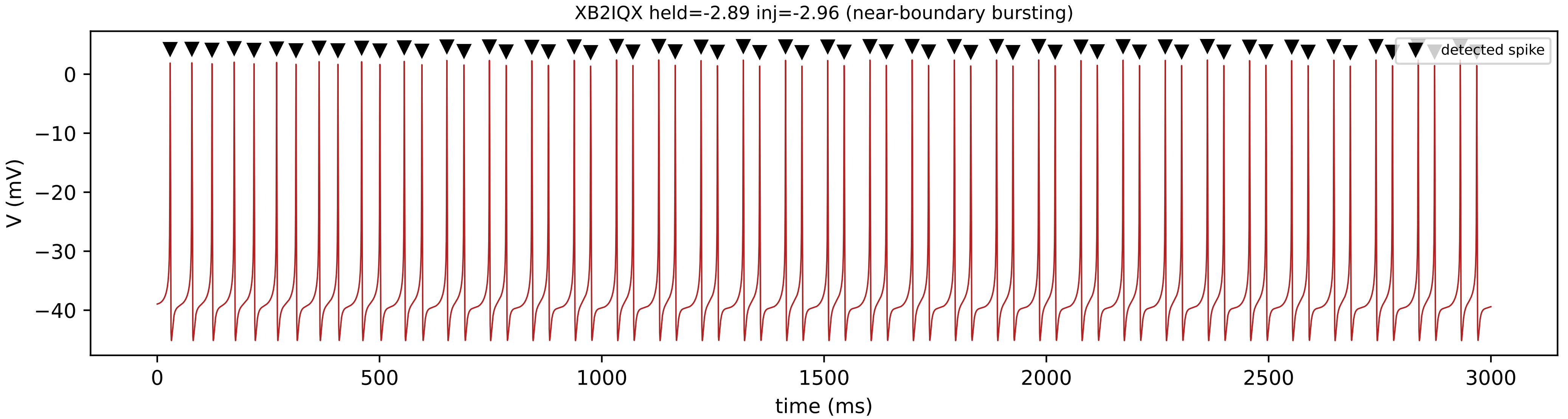
python src/generate_validation_packet.py
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()

XB2IQX: tonic firing during the current step, then a sustained (tonic) rebound

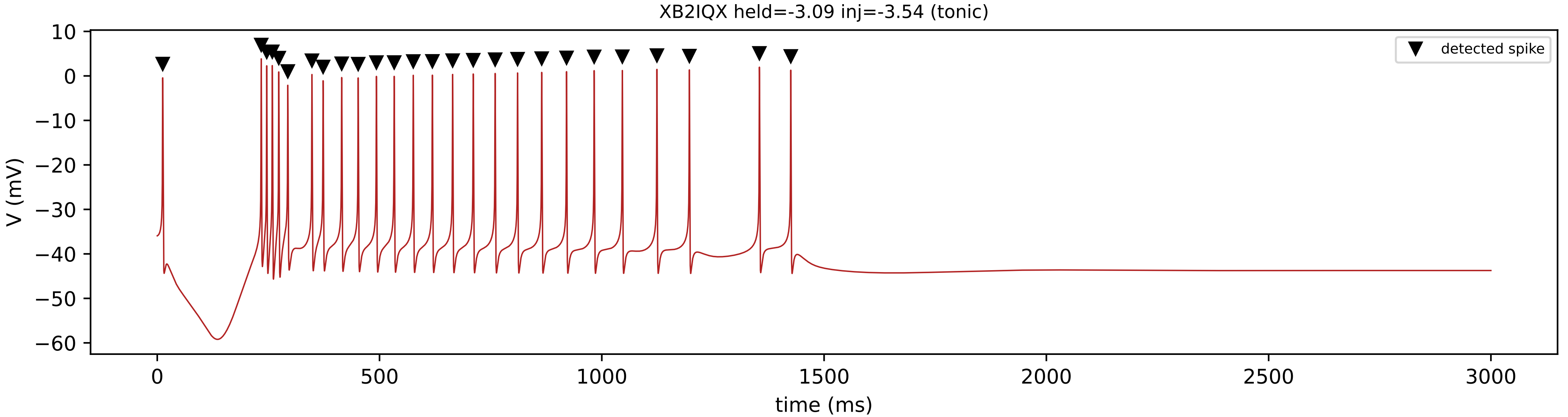


python src/generate_validation_packet.py
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()

2. Spike detection

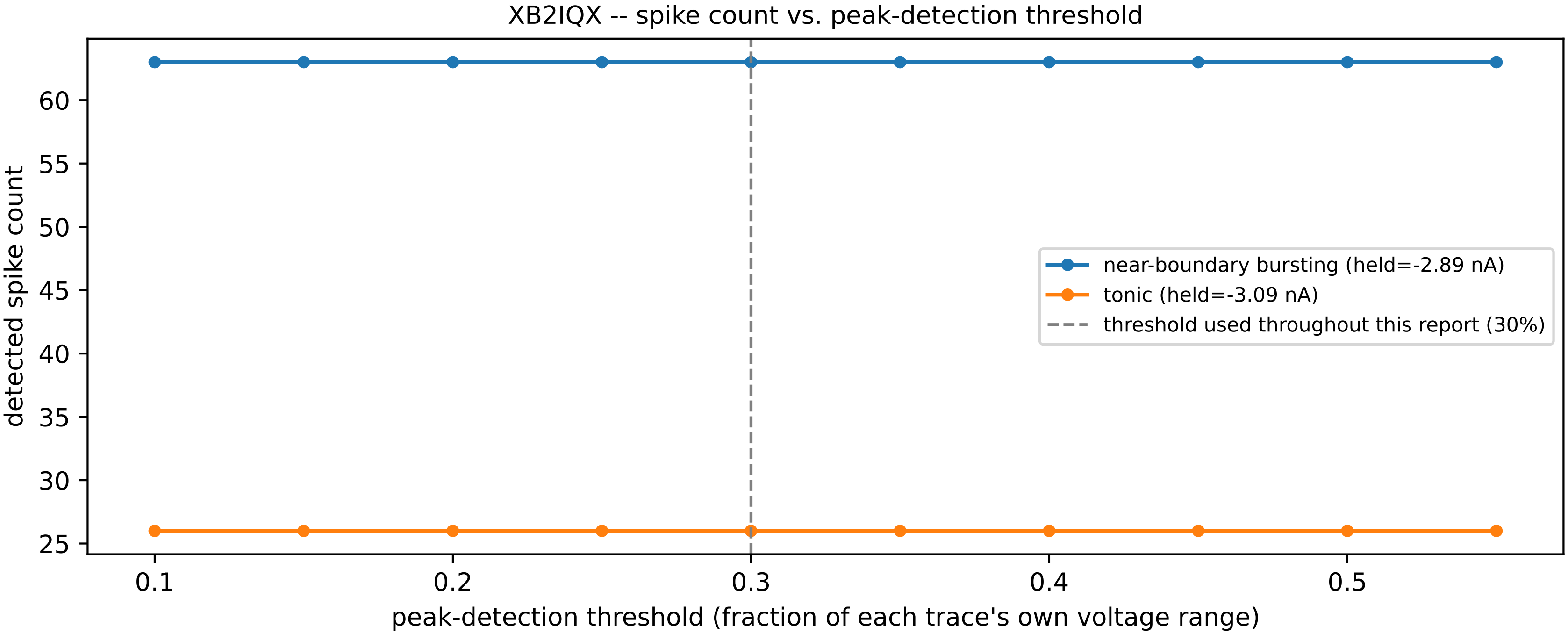


Markers show every spike this trace's automated peak detector found, requiring each peak to rise at least 15.0 mV above its surroundings (a fixed 15 mV floor, since this trace's own 47.6 mV voltage range is small enough that a simple 30% threshold would let subthreshold noise through) -- this is the same spike count used for every interspike-interval and burst/tonic classification elsewhere in this report. 63 spike(s) were detected in this window.



python src/generate_validation_packet.py
source: cell_held_injected_grid.pkl -> resimulate_point() -> trace_annotations.mark_spikes() (scipy.signal.find_peaks, same call find_silencing_threshold.compute_isis_ms uses)

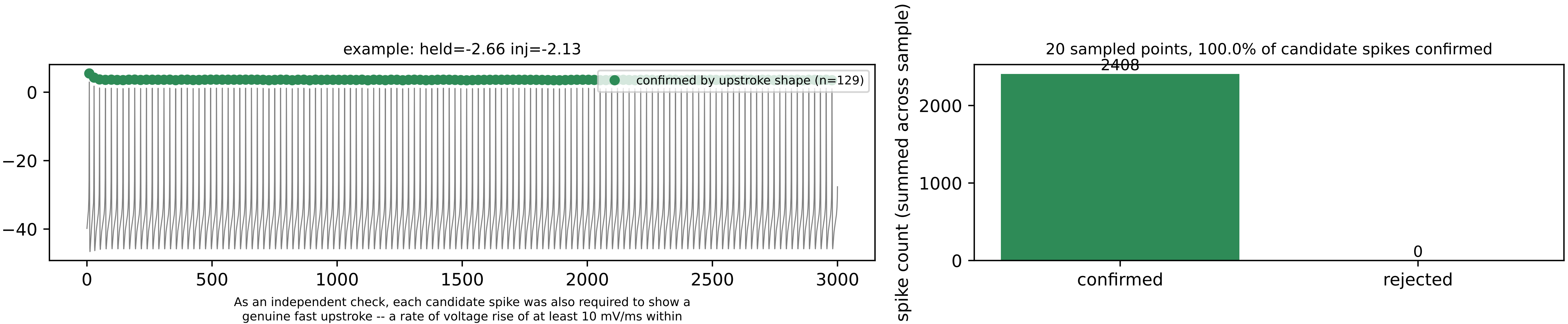
3. Spike-detection threshold sensitivity



Every spike count, interspike interval, and burst/tonic classification in this report depends on one detection threshold: a candidate peak must rise at least 30% of the trace's own voltage range above its surroundings (or 15 mV, whichever is larger, so a tiny subthreshold wobble in a low-amplitude trace cannot be counted as a spike). This sweeps that threshold from 10% to 55% on real, large-amplitude spiking traces: a flat plateau around the value used throughout this report shows the reported spike counts are not sensitive to the exact threshold chosen, rather than sitting on an arbitrary cliff edge.

python src/generate_validation_packet.py
source: cell_held_injected_grid.pkl -> resimulate_point() -> scipy.signal.find_peaks swept over PROMINENCE_FRACTION

4. Independent cross-check of spike detection by upstroke shape

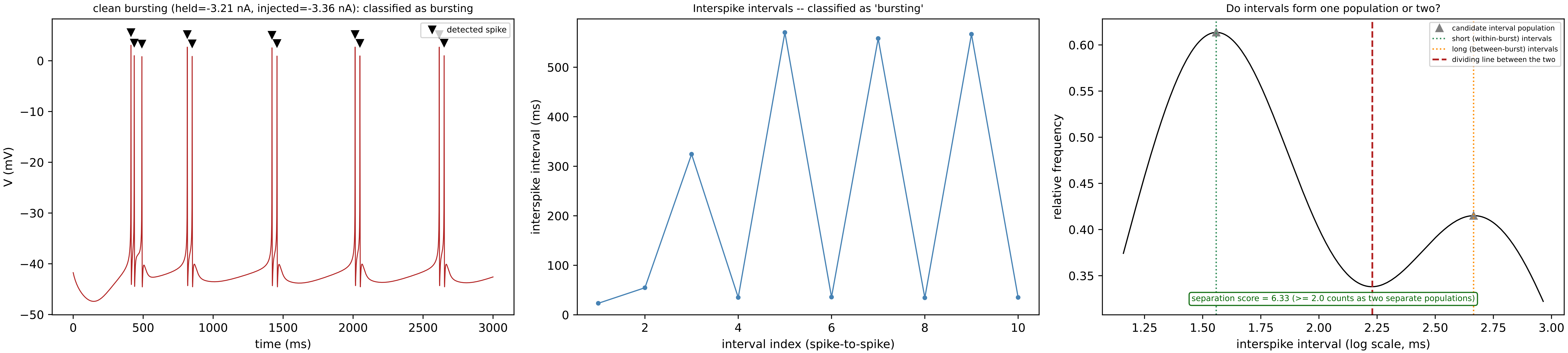


As an independent check, each candidate spike was also required to show a genuine fast upstroke -- a rate of voltage rise of at least 10 mV/ms within 20 ms before the peak (the convention used by Bean, 2007) -- rather than amplitude alone. 129 of 129 candidate spike(s) were confirmed by this stricter test; the two methods agree on every spike in this window.

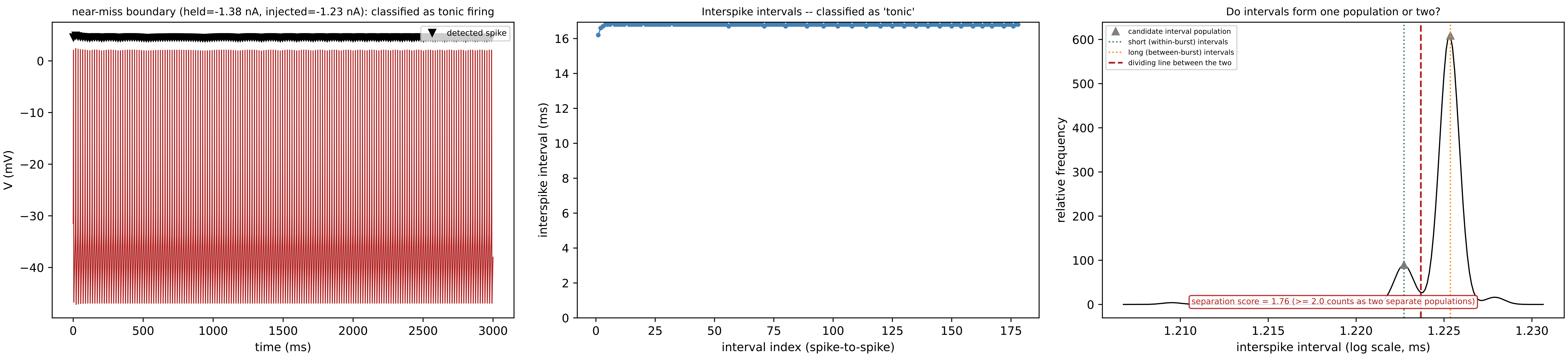
As an independent check on a random sample of 20 grid points from this cell, every spike detected by amplitude alone was additionally required to show a genuine fast upstroke in voltage (rather than amplitude alone) before being counted, an independent detection criterion not used for classification elsewhere in this report. Agreement close to 100% shows that the simpler amplitude-based detector used throughout is not missing any spikes that a shape-based check would catch.

python src/generate_validation_packet.py
source: cell_held_injected_grid.pkl -> resimulate_point() -> find_silencing_threshold.detect_spikes_dvdt_confirmed()

5. Tonic / bursting / silent classification



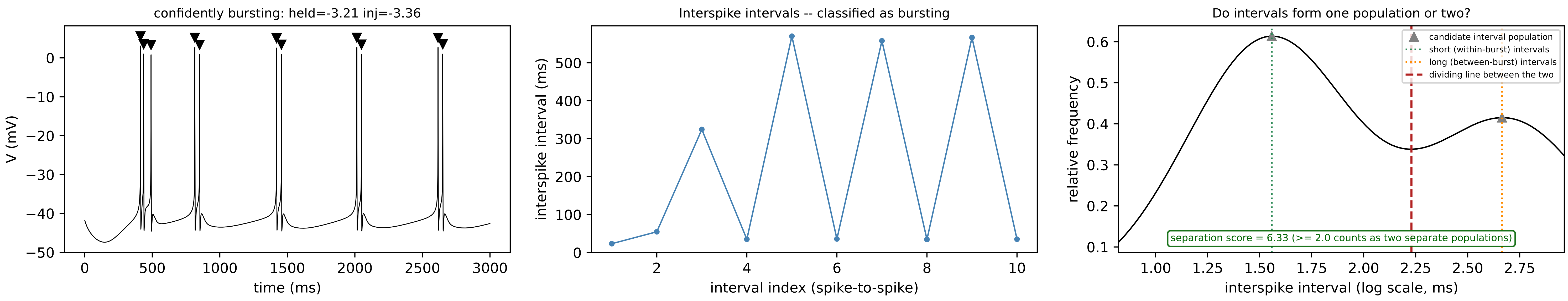
This window contained 11 spikes, giving 10 interspike intervals. Splitting the intervals at their 2 candidate population(s) gives a short (within-burst) interval of 36.4 ms and a long (between-burst) interval of 504.8 ms -- a 13.88-fold difference, above the 1.5-fold difference required to treat them as distinct. On average, 2.20 spikes fall within each burst (a real burst requires at least 1.5, ruling out what would otherwise just be occasional missed spikes in an otherwise tonic train). A statistical separation score (Ashman's D) of 6.33 quantifies how cleanly the short and long intervals separate into two distinct populations, rather than overlapping as one broad distribution (a score of 2.0 or higher is required to call them separate). Taken together, this window is classified as 'bursting', with a separation score of 6.33.



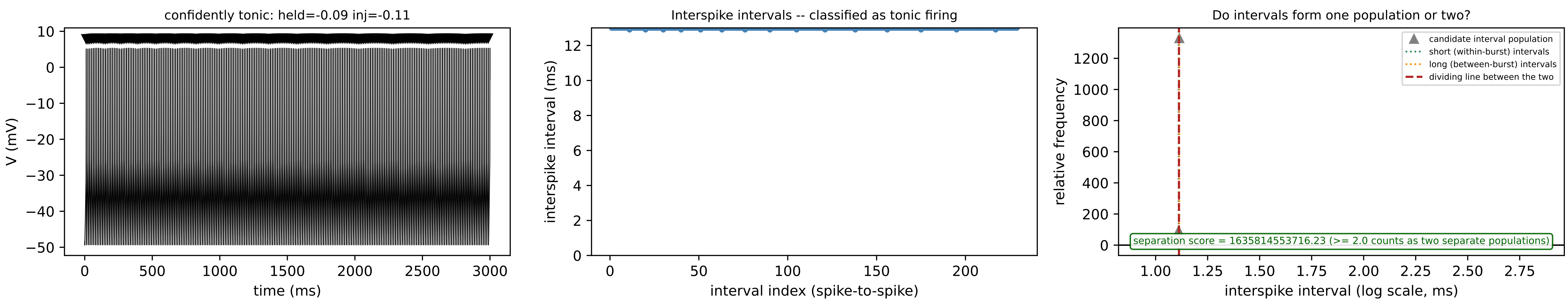
This window contained 179 spikes, giving 178 interspike intervals. Splitting the intervals at their 2 candidate population(s) gives a short (within-burst) interval of 16.7 ms and a long (between-burst) interval of 16.8 ms -- a 1.01-fold difference, above the 1.5-fold difference required to treat them as distinct. A statistical separation score (Ashman's D) of 1.76 quantifies how cleanly the short and long intervals separate into two distinct populations, rather than overlapping as one broad distribution (a score of 2.0 or higher is required to call them separate). Taken together, this window is classified as 'tonic', with a separation score of 1.76.

python src/generate_validation_packet.py
source: cell_held_injected_grid.pkl -> resimulate_point() -> find_silencing_threshold.classify_burst_pattern()

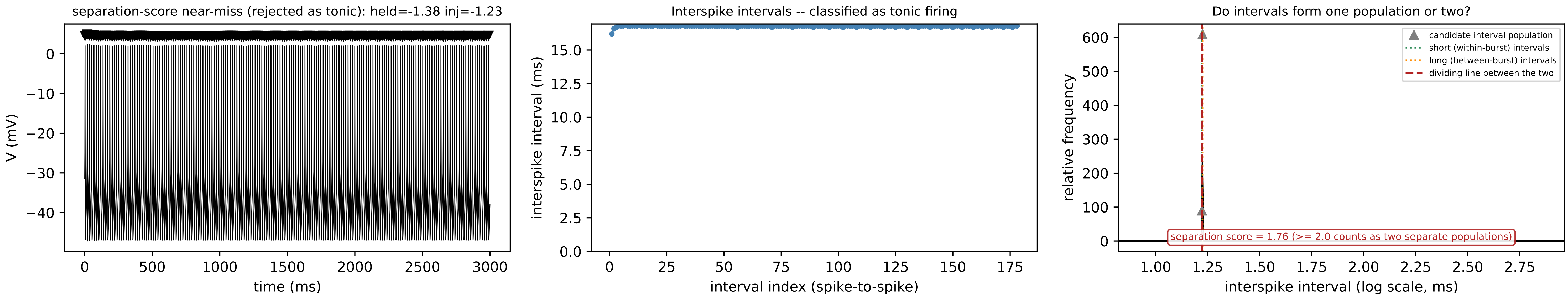
6. Bimodality test: within-burst vs. between-burst ISIs



Simulated XB2IQX trace. This window contained 11 spikes, giving 10 interspike intervals. Splitting the intervals at their 2 candidate population(s) gives a short (within-burst) interval of 36.4 ms and a long (between-burst) interval of 504.8 ms -- a 13.88-fold difference, above the 1.5-fold difference required to treat them as distinct. On average, 2.20 spikes fall within each burst (a real burst requires at least 1.5, ruling out what would otherwise just be occasional missed spikes in an otherwise tonic train). A statistical separation score (Ashman's D) of 6.33 quantifies how cleanly the short and long intervals separate into two distinct populations, rather than overlapping as one broad distribution (a score of 2.0 or higher is required to call them separate). Taken together, this window is classified as 'bursting', with a separation score of 6.33.

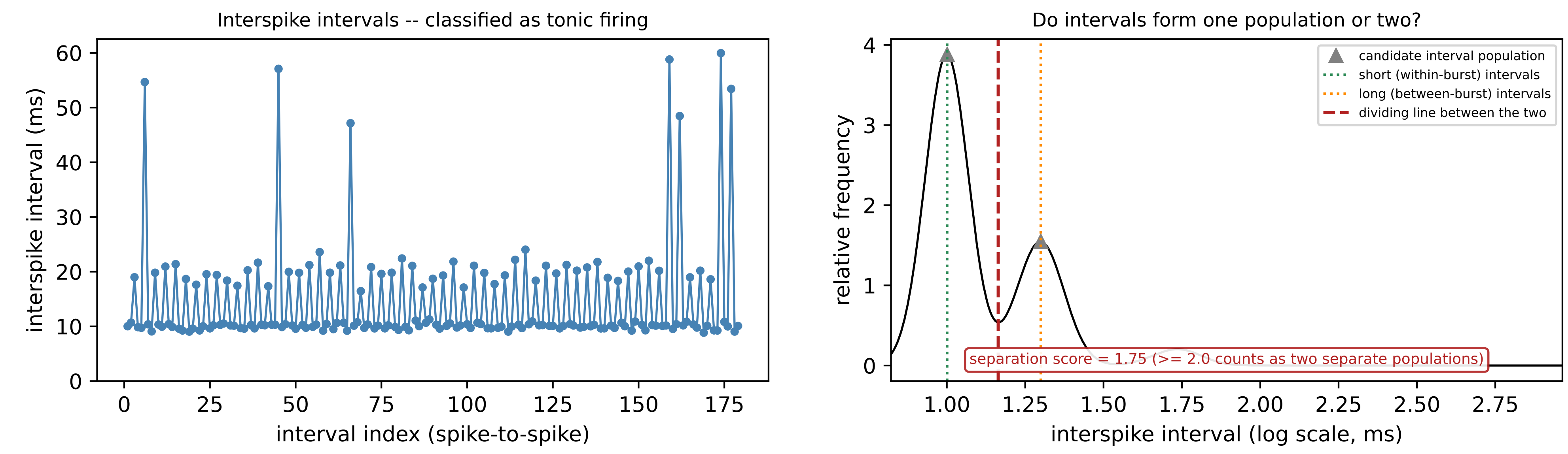


Simulated XB2IQX trace. This window contained 230 spikes, giving 229 interspike intervals. Splitting the intervals at their 2 candidate population(s) gives a short (within-burst) interval of 12.9 ms and a long (between-burst) interval of 13.0 ms -- a 1.01-fold difference, above the 1.5-fold difference required to treat them as distinct. A statistical separation score (Ashman's D) of 1635814553716.23 quantifies how cleanly the short and long intervals separate into two distinct populations, rather than overlapping as one broad distribution (a score of 2.0 or higher is required to call them separate). Taken together, this window is classified as 'tonic', with a separation score of 1635814553716.23.



Simulated XB2IQX trace. This window contained 179 spikes, giving 178 interspike intervals. Splitting the intervals at their 2 candidate population(s) gives a short (within-burst) interval of 16.7 ms and a long (between-burst) interval of 16.8 ms -- a 1.01-fold difference, above the 1.5-fold difference required to treat them as distinct. A statistical separation score (Ashman's D) of 1.76 quantifies how cleanly the short and long intervals separate into two distinct populations, rather than overlapping as one broad distribution (a score of 2.0 or higher is required to call them separate). Taken together, this window is classified as 'tonic', with a separation score of 1.76.

Constructed example (a hand-built interval sequence, not a simulated trace)

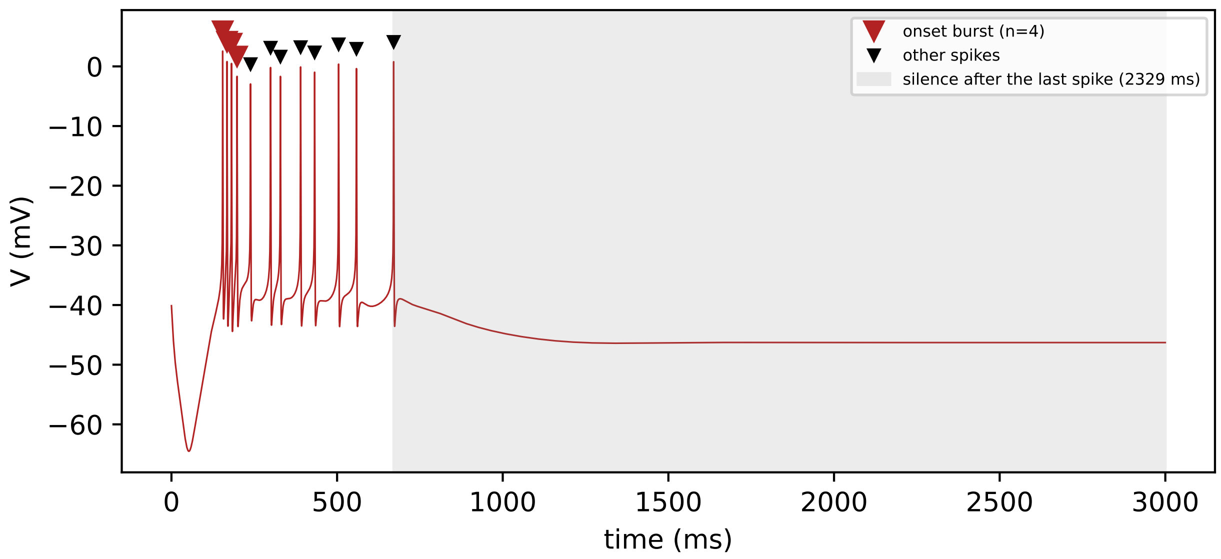


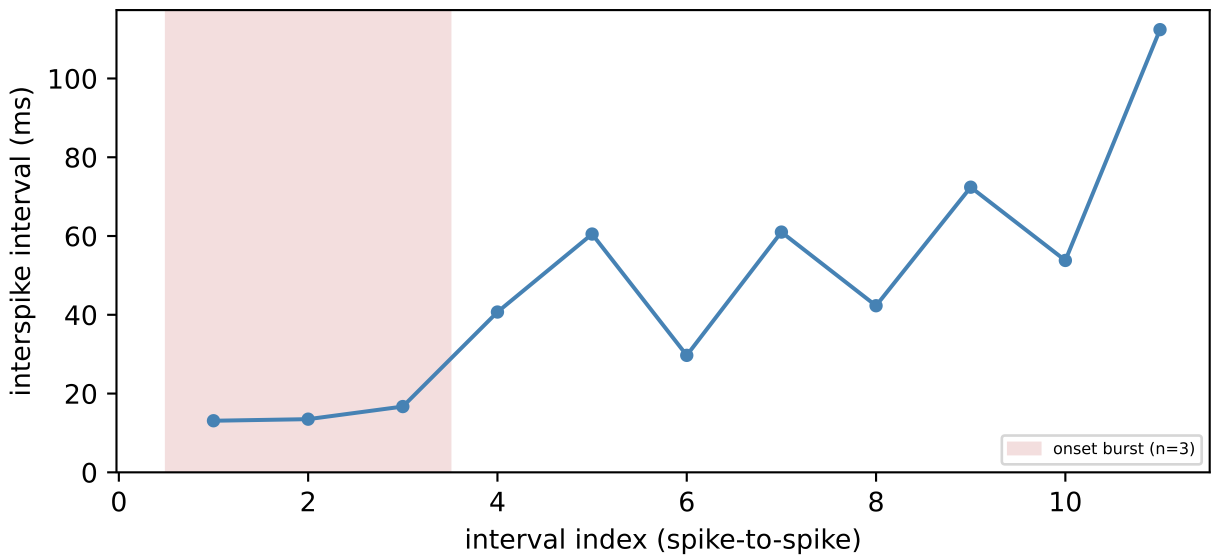
This example is constructed, not an observed point from XB2IQX: it illustrates a case where the short and long interval populations pass the simpler gates but remain too statistically overlapping to count as genuinely separate, a case this cell's own data has not been observed to produce. This window contained 180 spikes, giving 179 interspike intervals. Splitting the intervals at their 2 candidate population(s) gives a short (within-burst) interval of 10.0 ms and a long (between-burst) interval of 24.0 ms -- a 2.40-fold difference, above the 1.5-fold difference required to treat them as distinct. On average, 3.00 spikes fall within each burst (a real burst requires at least 1.5, ruling out what would otherwise just be occasional missed spikes in an otherwise tonic train). A statistical separation score (Ashman's D) of 1.75 quantifies how cleanly the short and long intervals separate into two distinct populations, rather than overlapping as one broad distribution (a score of 2.0 or higher is required to call them separate). Taken together, this window is classified as 'tonic', with a separation score of 1.75.

```
python src/generate_validation_packet.py
source: cell_held_injected_grid.pkl -> resimulate_point() -> find_silencing_threshold.classify_burst_pattern() (4th row: constructed synthetic ISIs, no resimulation)
```

7. Burst-onset-then-silence detection

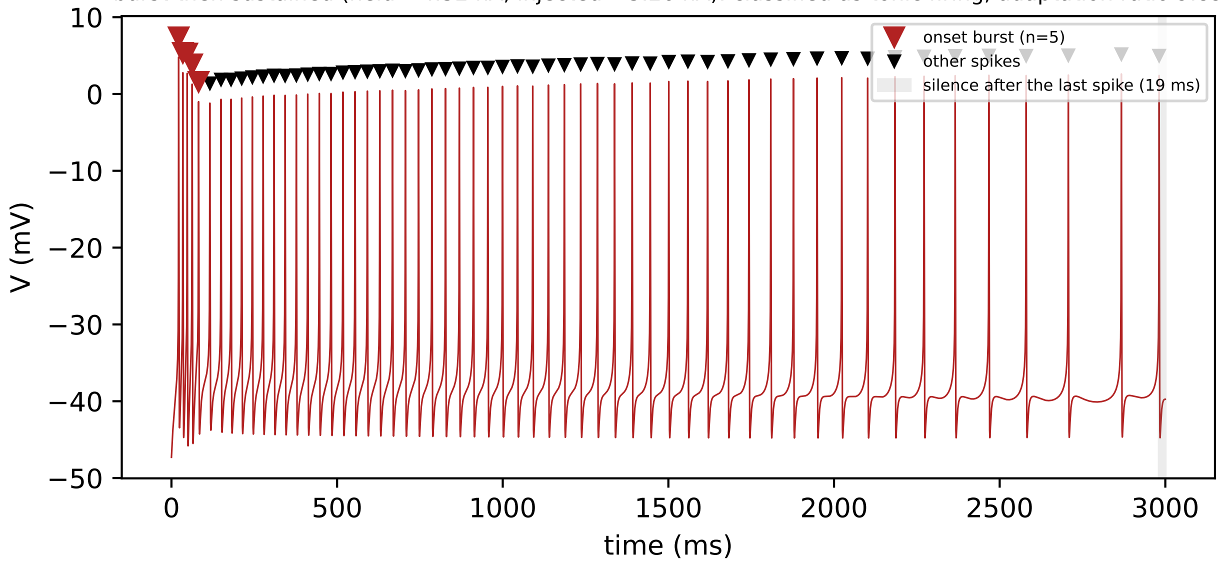
burst then ceased (held=-2.48 nA, injected=-4.04 nA): classified as bursting, adaptation ratio not reported (see caption)

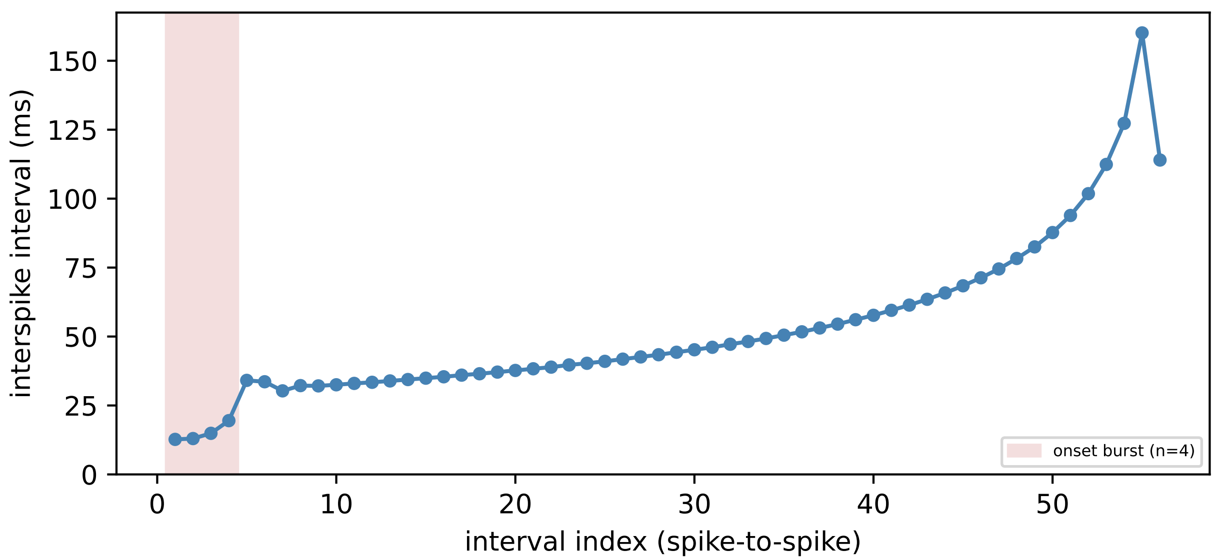




A leading onset burst of 4 spikes (mean interval 14.4 ms) occurred at the start of the window. This local test is independent of whichever tonic/bursting/silent label the whole-window statistical test assigns, so it can detect a real leading burst even when the window as a whole reads as tonic. Firing was followed by 2329 ms of silence before the end of this 3000 ms window. This silence is at least 3 times the most recent interspike interval, indicating firing had genuinely ended rather than merely being cut off by the window boundary; an adaptation ratio is not reported for this window, since comparing early and late intervals is not meaningful once firing has stopped partway through.

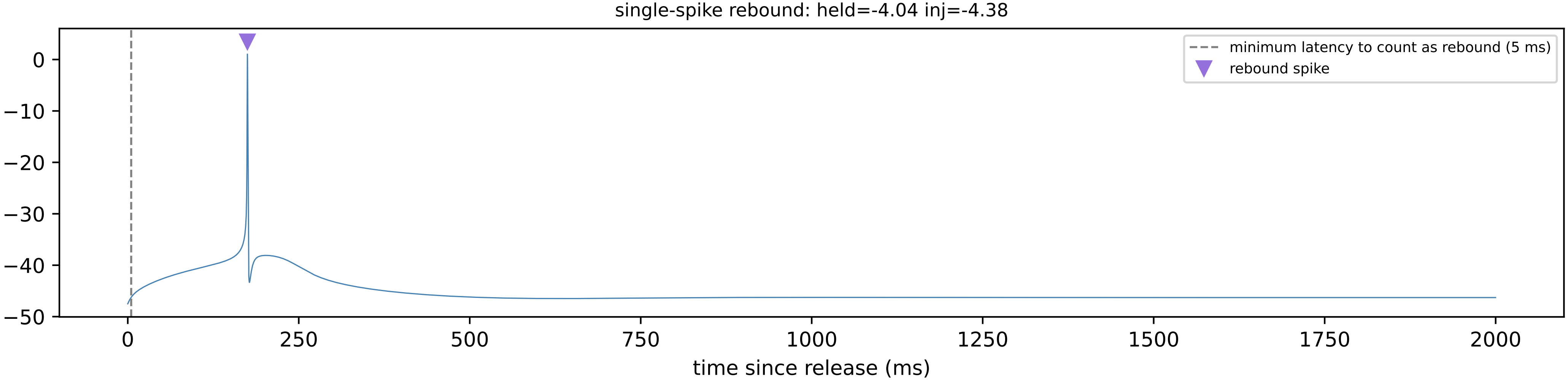
burst then sustained (held=-4.32 nA, injected=-3.26 nA): classified as tonic firing, adaptation ratio 9.89



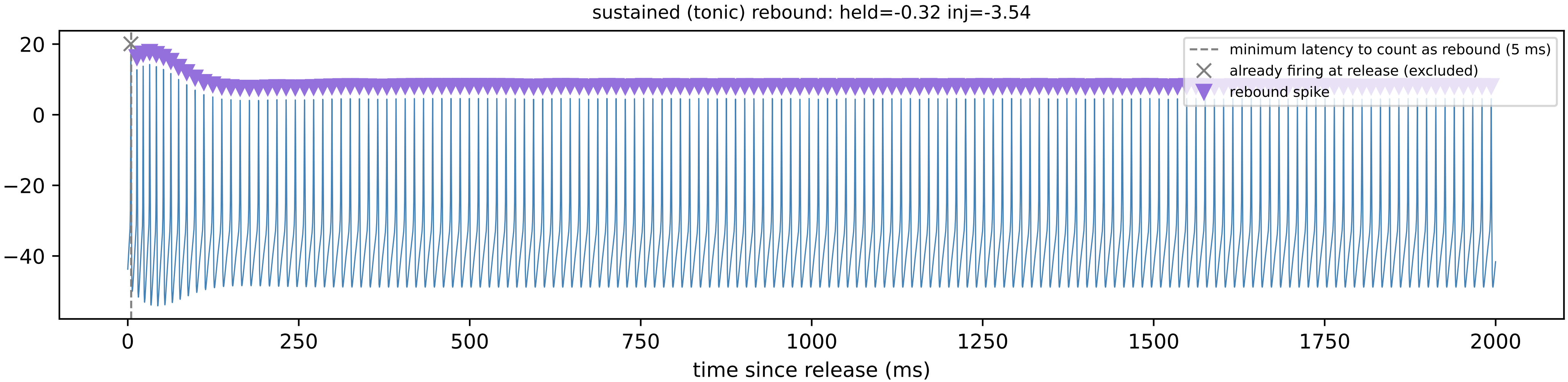


python src/generate_validation_packet.py
source: cell_held_injected_grid.pkl -> resimulate_point() -> run_held_injected_grid.detect_onset_burst()

8. Post-inhibitory rebound detection

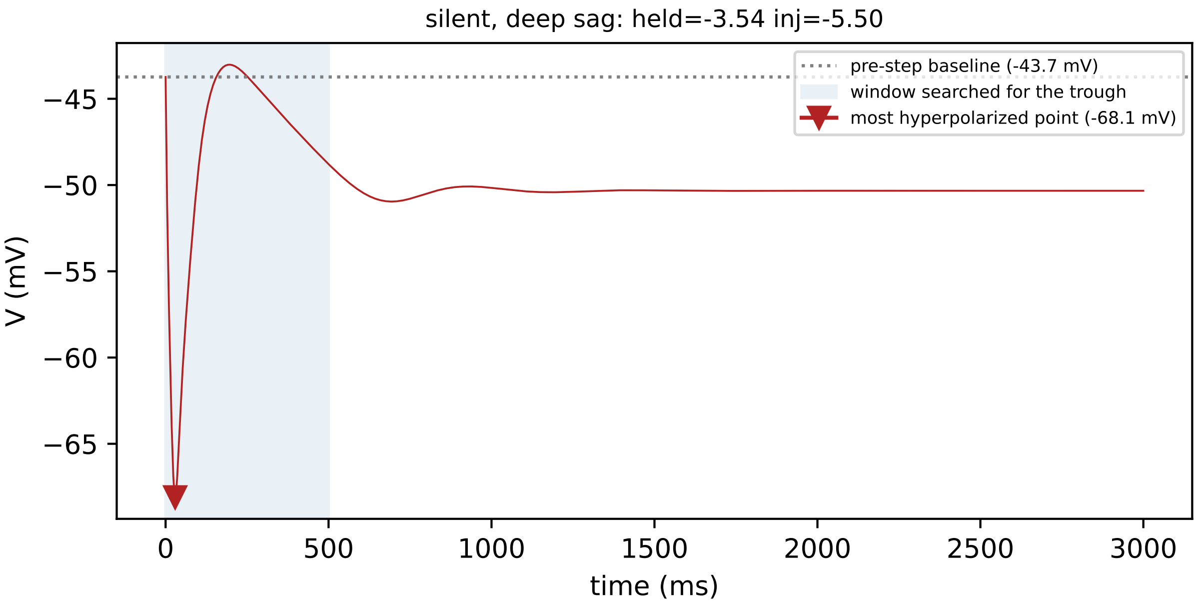


1 spike(s) occurred at or after 5 ms post-release and so count as post-inhibitory rebound. This cell did show a rebound response.



python src/generate_validation_packet.py
source: cell_held_injected_grid.pkl -> resimulate_point() -> trace_annotations.mark_rebound_window() (reproduces run_held_injected_grid.run_test_and_recovery's rebound-peak logic)

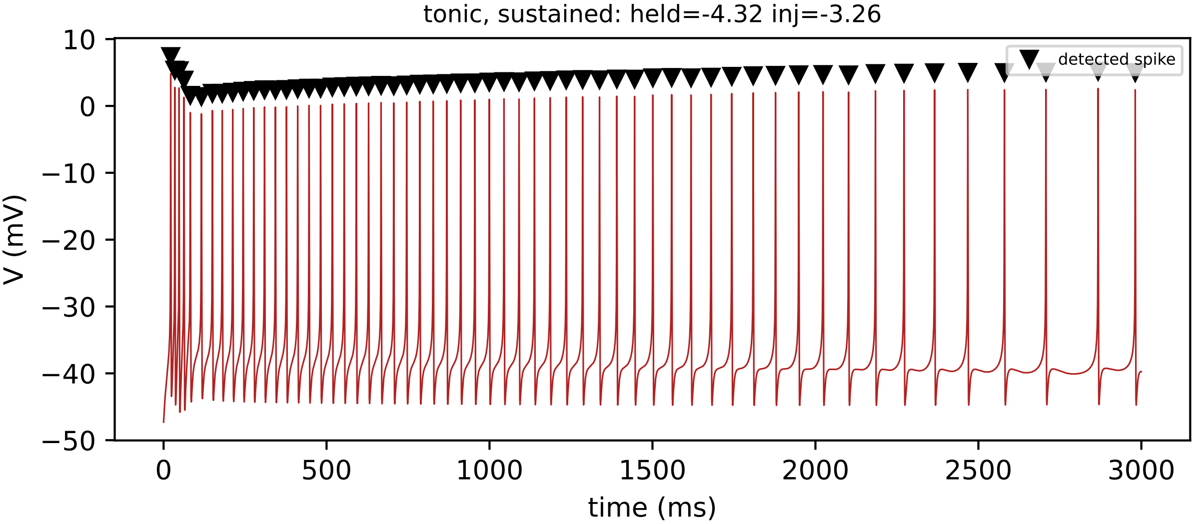
9. Sag depth

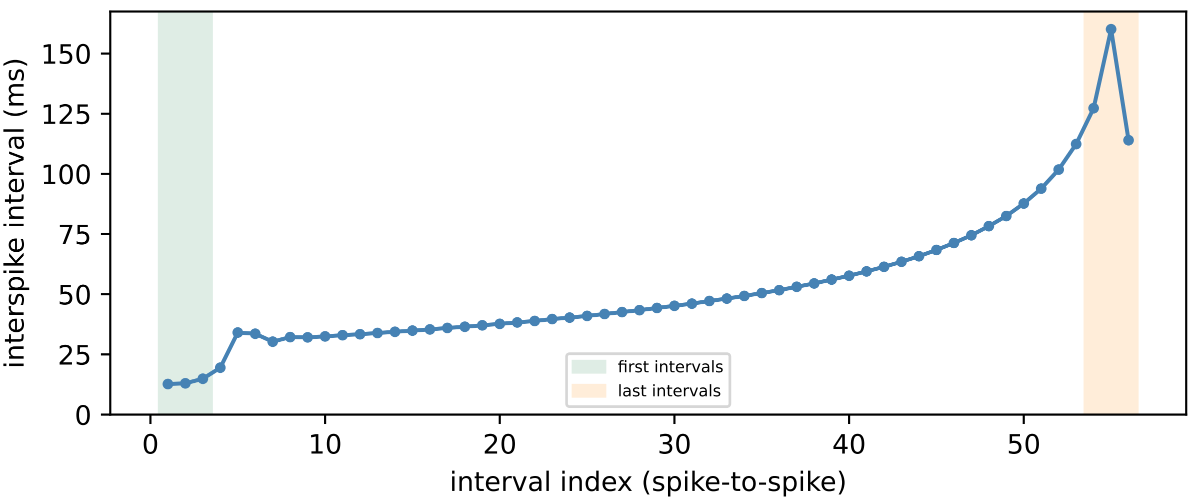


Sag depth is the difference between the voltage just before the current step (-43.7 mV) and the most hyperpolarized point reached afterward (-68.1 mV), giving 24.4 mV of sag -- a hallmark of the hyperpolarization-activated current (Ih) partially repolarizing the cell during the step. The trough was found by searching the full 500 ms window (no spike occurred before it ended).

python src/generate_validation_packet.py
source: cell_held_injected_grid.pkl -> resimulate_point() -> run_held_injected_grid.compute_pre_spike_sag_trough()

10. Spike-frequency adaptation

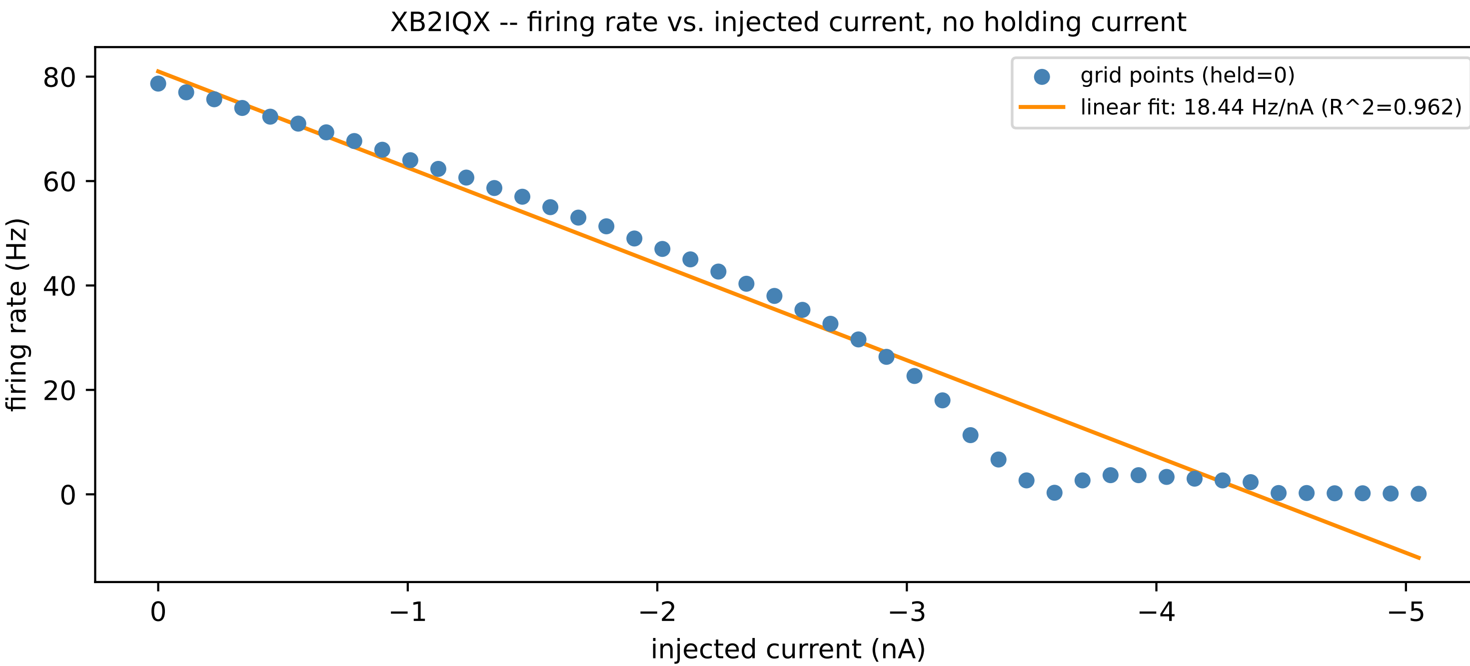




The adaptation ratio -- the mean of the last 3 interspike intervals divided by the mean of the first 3 -- is 9.89. Firing slows over the course of firing (spike-frequency adaptation) over this window.

```
python src/generate_validation_packet.py
source: cell_held_injected_grid.pkl -> resimulate_point() -> run_held_injected_grid.compute_adaptation_ratio()
```

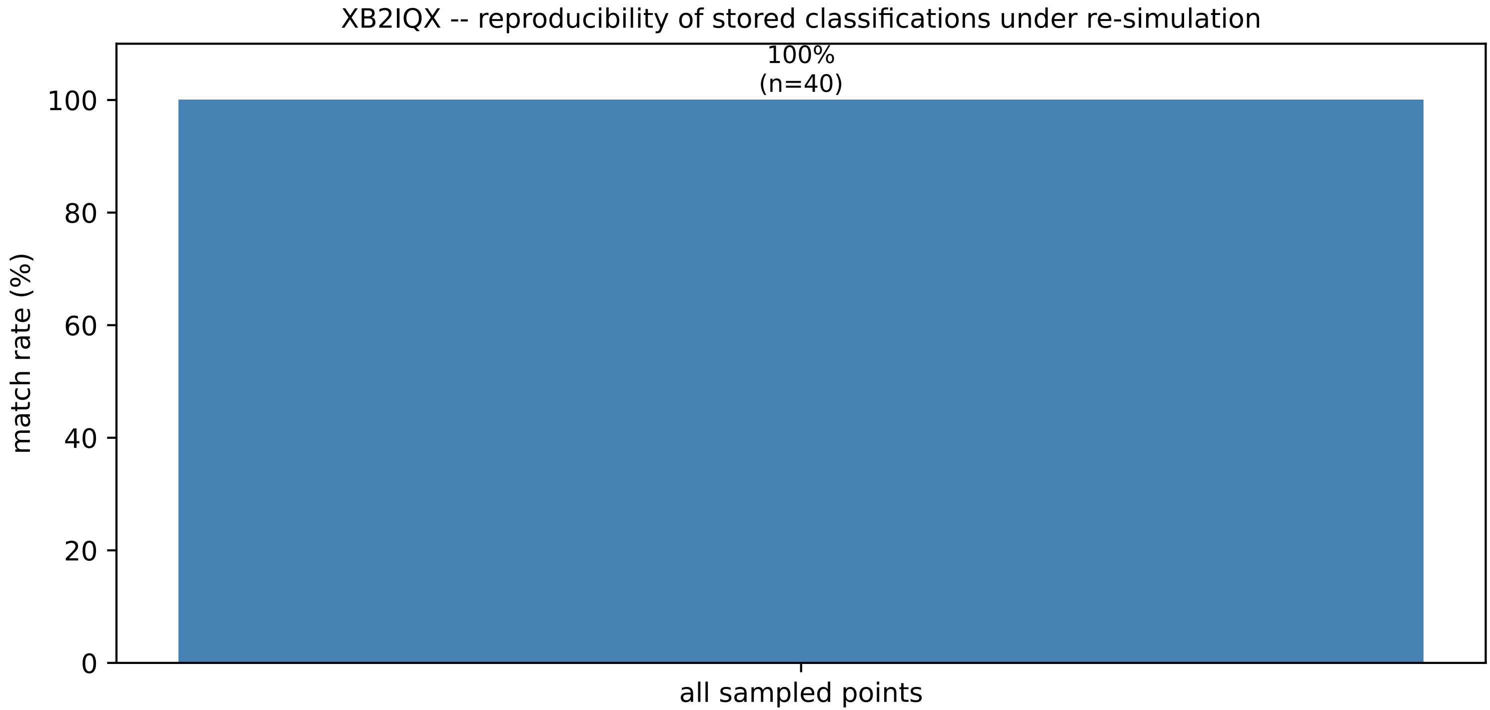
11. Firing rate vs. injected current



46 non-silent points with a clearly-classified firing pattern, sampled across injected current with no holding current applied (points still below firing threshold, which would otherwise bias the linear fit toward zero, are excluded). This firing-rate/current slope is one of the features used to characterize this cell in the cross-cell comparison; here it is plotted directly against the data it was fit to.

python src/generate_validation_packet.py
source: cell_held_injected_grid.pkl -> cell_grid_features.pkl (extract_grid_features.compute_fi_slope())

12. Reproducibility check



40 random points from this cell's grid were re-simulated independently and compared against the classification stored from the original sweep. Match rate: 100%. Except at zero holding current, where the cell's own steady-state limit cycle is reused exactly, each holding-current level is reached by integrating forward from whichever already-settled level is numerically nearest -- this model is known to show path-dependent (hysteretic) settling near dynamical transitions, so a re-simulation that approaches a borderline point along a different path than the original sweep can occasionally land on a different classification right at that boundary. Any mismatches here are consistent with that known property of the model rather than indicating an error in the classification code.

```
python src/generate_validation_packet.py
source: cell_held_injected_grid.pkl -> cached test_pattern/rebound_pattern vs. fresh resimulate_point() results
```